Git2: Branches und Remotes

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Neues Feature entwickeln/ausprobieren

```
A---B---C  master
```

Neues Feature entwickeln/ausprobieren

```
A---B---C  master
```

1. `git branch wuppie`
2. `git checkout wuppie` oder `git switch wuppie`

```
A---B---C  master, wuppie
```

...

```
          D  wuppie
         /
A---B---C  master
```

Konsole

Problem: Fehler im ausgelieferten Produkt

1. `git checkout master`
2. `git checkout -b fix`
3. Änderungen in `fix` vornehmen ...

```
           D  wuppie                                  D  wuppie
          /                                          /
 A---B---C  master              =>        A---B---C  master
                                                     \
                                                      E  fix
```

Fix ist stabil: Integration in *master*

1. `git checkout master`
2. `git merge fix` => **fast forward** von `master`
3. `git branch -d fix`

```
          D  wuppie                                  D  wuppie
         /                                          /
A---B---C  master              =>        A---B---C---E  master
         \
          E  fix
```

Feature weiter entwickeln und Integration in *master*

1. `git checkout master`
2. `git merge wuppie`
   => Kein *fast forward* möglich: Git sucht nach gemeinsamen Vorgänger + neuer Commit

```
          D---F  wuppie                            D---F  wuppie
         /                      =>                /     \
A---B---C---E  master                    A---B---C---E---G  master
```

Konflikte beim Mergen

```
$ git merge wuppie
Auto-merging hero.java
CONFLICT (content): Merge conflict in hero.java
Automatic merge failed; fix conflicts and then commit the result.
```

Konflikte beim Mergen

```
$ git merge wuppie
Auto-merging hero.java
CONFLICT (content): Merge conflict in hero.java
Automatic merge failed; fix conflicts and then commit the result.
```

```
<<<<<<< HEAD:hero.java
public void getActiveAnimation() {
    return null;
=======
public Animation getActiveAnimation() {
    return this.idleAnimation;
>>>>>>> wuppie:hero.java
```

Konsole: Branchen und Mergen

Rebasen: Verschieben von Branches

```
          D---F  wuppie                          D---F  wuppie
         /                    =>                /     \
A---B---C---E  master                    A---B---C---E---G  master
```

Rebasen: Verschieben von Branches

```
          D---F  wuppie                          D---F  wuppie
         /                      =>              /     \
A---B---C---E  master                    A---B---C---E---G  master
```

```
git rebase master wuppie
```

```
                D'---F'  wuppie
               /
A---B---C---E  master
```

Erinnerung: Clonen kann sich lohnen => Remote-Branches

```
https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture

---C---D---E  master
```

=> `git clone https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture`

```
./Prog2-Lecture/  (lokaler Rechner)

---C---D---E  master
           ^origin/master
```

Eigener und entfernter *master* entwickeln sich weiter . . .

```
https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture

---C---D---E---F---G  master
```

```
./Prog2-Lecture/  (lokaler Rechner)

---C---D---E---H  master
           ^origin/master
```

Änderungen vom Remote holen und Branches lokal zusammenführen

```
https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture

---C---D---E---F---G  master
```

=> `git fetch origin`

```
./Prog2-Lecture/  (lokaler Rechner)

---C---D---E---H  master
           \
            F---G  origin/master
```

Beispiel für Zusammenführen (merge und push), Anmerkung zu fast forward merge

Wrap-Up

- Anlegen von Branches mit `git branch`
- Umschalten der Workingcopy auf anderen Branch: `git checkout` oder `git switch`
- Mergen von Branches und Auflösen von Konflikten: `git merge`
- Verschieben von Branches mit `git rebase`
- Änderungen vom Server holen: `git fetch <remote>`
- Lokalen Branch auffrischen: `git merge <remote>/<branch>`
- Eigene Änderungen hochladen: `git push <remote> <branch>`

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.